Performance Evaluation of Optimised Dense and Sparse Matrix Multiplication in Python

Controlled Synthetic Experiments and the `mc2depi` Case Study

Javier Bolívar García-Izquierdo
Degree programme: [COMPLETE]
Universidad de Las Palmas de Gran Canaria
Course: [COMPLETE]

28 October 2026

Project version: Experimental implementation developed for Individual Assignment 2

Programming language: Python 3.13.12

Dataset: Williams/`mc2depi`

Contents

1 Introduction **4**
- 1.1 Research Questions . . . 4
- 1.2 Hypotheses . . . 4

2 Experimental Environment **5**
- 2.1 Hardware and Operating System . . . 5
- 2.2 Software Environment . . . 5
- 2.3 Measurement Conditions . . . 5

3 Methodology and Implementation **5**
- 3.1 Dense Implementations . . . 5
- 3.2 Sparse Implementations . . . 6
- 3.3 Correctness Validation . . . 6
- 3.4 Synthetic Input Design . . . 7

4 Dense Benchmark Results **7**
- 4.1 Execution Time . . . 7
- 4.2 Memory Usage . . . 9

5 Sparse Benchmark Results **9**
- 5.1 Execution Time and Speedup . . . 9
- 5.2 Product Densification . . . 11
- 5.3 Memory Usage . . . 12

6 Real-World Case Study: `mc2depi` **12**
- 6.1 Dataset Characteristics . . . 12
- 6.2 Sparse Matrix-Vector Multiplication . . . 13
- 6.3 Sparse Matrix-Matrix Multiplication . . . 13

7 Discussion **14**
- 7.1 Dense Optimisation Behaviour . . . 14
- 7.2 Sparse Performance and Crossover Behaviour . . . 14
- 7.3 Importance of Output Structure . . . 14
- 7.4 Threats to Validity . . . 14

8 Conclusions **15**
- 8.1 Future Work . . . 15

9 AI-Assistance Declaration **16**

A Reproduction Commands **16**

B Generated Figures **17**

Abstract

This study evaluates the execution-time and memory behaviour of dense and sparse matrix multiplication implementations in Python. The dense study compares a conventional triple-loop implementation using the `i-j-k` loop order with a row-oriented `i-k-j` implementation and a tiled implementation using blocks of 32 elements. The sparse study compares the row-oriented dense implementation with a custom Compressed Sparse Row implementation and SciPy CSR multiplication using identical logical input matrices.

Synthetic square matrices with side lengths from 64 to 512 were used to examine scaling behaviour. Sparse experiments considered input sparsity levels of 50%, 70%, 90%, 95% and 99%. Every final configuration included one warm-up execution and five measured executions. Execution time was summarised using the median and interquartile range, while process memory was measured in independent Python processes using sampled Resident Set Size.

The row-oriented dense implementation produced the best dense execution time at the largest tested size, whereas the tiled Python implementation did not compensate for its additional loop overhead. Sparse methods became increasingly advantageous as input sparsity increased, although the experiments also demonstrated that sparse inputs can produce an almost dense result. SciPy CSR consistently provided the strongest sparse performance in the larger controlled configurations.

The real-world `mc2depi` matrix contains 525,825 rows, 525,825 columns and 2,100,225 stored nonzero entries. Its sparsity is approximately 99.99924040%. The matrix required approximately 26.041 MiB for its CSR arrays, whereas an equivalent dense `int64` value array would theoretically require approximately 2,060.025 GiB. The final sparse matrix-matrix product was completed with a median execution time of 0.0641652 seconds and produced 5,245,952 nonzero entries.

Keywords: matrix multiplication, benchmark, dense matrix, sparse matrix, CSR, SciPy, performance, memory, `mc2depi`.

1 Introduction

Matrix multiplication is a fundamental operation in scientific computing, numerical simulation, graph processing and data analysis. Its computational and memory costs depend on matrix dimensions, storage representation, implementation strategy and the distribution of nonzero values.

A dense matrix representation stores every logical position, including zeros. This representation provides direct indexing and predictable traversal, but its storage requirement grows proportionally to the total number of matrix positions. Sparse representations store only selected values and their structural indices. They can substantially reduce storage and unnecessary arithmetic when zeros dominate the inputs, although indexing overhead and irregular memory access may reduce their advantage.

This study evaluates both dense optimisation techniques and sparse matrix representations. The first part investigates whether loop reordering and tiling improve the performance of a conventional Python matrix multiplication implementation. The second part investigates how matrix size and sparsity affect the relative performance of dense multiplication, a custom CSR implementation and SciPy CSR multiplication.

The controlled synthetic experiments are complemented by a real-world case study based on `mc2depi`, an epidemiology matrix from the SuiteSparse Matrix Collection. The real-world case demonstrates why a sparse representation is required for a matrix whose equivalent dense representation would exceed the memory capacity of the experimental system.

1.1 Research Questions

The study addresses the following research questions:

RQ1. How do row-oriented and tiled traversal affect dense matrix multiplication time and memory usage as matrix size increases?

RQ2. Under which tested combinations of matrix size and input sparsity do CSR representations outperform the matched dense implementation?

RQ3. How does input sparsity affect the sparsity of the multiplication result?

RQ4. What execution-time and storage characteristics are observed when processing the real-world `mc2depi` dataset?

1.2 Hypotheses

- **H1:** Row-oriented traversal is expected to reduce execution time relative to the basic implementation by providing more sequential access to rows of the second input matrix.
- **H2:** Tiling is expected to improve data locality, although its benefit may be limited by Python loop overhead.
- **H3:** CSR multiplication is expected to become increasingly advantageous as input sparsity increases.
- **H4:** High input sparsity is not expected to guarantee equally high output sparsity.

- **H5:** CSR storage is expected to make `mc2depi` feasible to process without constructing its dense equivalent.

2 Experimental Environment

2.1 Hardware and Operating System

The experiments were performed on an HP Victus by HP Gaming Laptop 15-fa2xxx. The system contained a 13th Generation Intel Core i5-13420H processor with eight physical cores and twelve logical processors. Windows reported a maximum clock value of 2100 MHz. The installed physical memory was 15.64 GiB.

The operating system was Microsoft Windows 11 Pro, version 10.0.26300, build 26300, using a 64-bit architecture. The active Windows power plan was Balanced.

2.2 Software Environment

The implementation used CPython 3.13.12 in a local virtual environment. The principal library versions were:

- NumPy 2.5.4;
- SciPy 1.18.1;
- pandas 3.1.0;
- Matplotlib 3.11.2;
- psutil 7.2.2;
- pytest 9.2.0.

2.3 Measurement Conditions

Each final configuration used one warm-up execution followed by five measured executions. Synthetic inputs were generated deterministically using recorded seeds, and all algorithms within a comparison received identical logical matrices.

Matrix generation and representation conversion were excluded from the timed multiplication interval. Execution time was measured with `time.perf_counter_ns()`. The principal summary statistics were the median and interquartile range.

Memory measurements were performed in fresh Python processes. Process Resident Set Size was sampled through `psutil` at intervals of 0.001 seconds. RSS includes the interpreter, imported modules, allocator state and matrix data; therefore, small RSS differences must not be interpreted as exact matrix-storage differences.

3 Methodology and Implementation

3.1 Dense Implementations

Three pure-Python dense multiplication implementations were evaluated.

3.1.1 Basic `i-j-k` Multiplication

The baseline algorithm follows the conventional triple-loop formulation:

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} B_{kj}.$$

The loop order is `i-j-k`. For each output position, the implementation traverses one row of the first matrix and one column of the second matrix.

3.1.2 Row-Oriented `i-k-j` Multiplication

The row-oriented implementation changes the loop order to `i-k-j`. For every value A_{ik}, the algorithm traverses row k of the second matrix and accumulates contributions into the current output row. This organisation provides sequential access to the stored Python rows of the second operand.

3.1.3 Tiled Multiplication

The tiled algorithm divides all three dimensions into blocks. A tile size of 32 was used in the final experiments. Partial tiles were supported when a matrix dimension was not an exact multiple of the block size.

3.2 Sparse Implementations

3.2.1 Custom CSR Representation

The custom Compressed Sparse Row representation uses three arrays:

- stored nonzero values;
- column indices associated with those values;
- row pointers identifying the range belonging to each row.

Sparse multiplication uses row accumulation. For every stored value A_{ik}, the algorithm traverses the stored entries in row k of the second matrix. Contributions are accumulated in a dictionary indexed by output column. Zero-valued cancellations are removed from the final representation.

3.2.2 SciPy CSR

The same logical inputs were converted to SciPy CSR matrices. Matrix multiplication was then performed using SciPy sparse matrix operations. Duplicate entries were combined, explicit zeros were removed and indices were sorted before collecting final metadata.

3.3 Correctness Validation

Correctness was validated before benchmarking. The test suite contained 102 successful tests covering:

- known square and rectangular products;
- identity and zero matrices;

- invalid dimensions and malformed inputs;
- deterministic random inputs;
- equivalence with NumPy;
- equivalence between the custom CSR implementation and SciPy CSR;
- sparsity, density and nonzero-count calculations;
- cancellation of values that result in zero.

Measured executions also checked result checksums and output nonzero counts. The final `mc2depi` matrix-matrix experiment additionally applied SHA-256 to the complete canonical CSR value, column-index and row-pointer arrays.

3.4 Synthetic Input Design

Dense square matrices used side lengths of 64, 128, 256 and 512. Their integer values were sampled from the interval $[-10, 10]$.

The sparse experiments used the same matrix sizes and requested sparsity levels of 50%, 70%, 90%, 95% and 99%. Nonzero positions were selected deterministically without replacement. Stored integer values were sampled from $[-10, 10]$, excluding zero.

4 Dense Benchmark Results

4.1 Execution Time

Figure 1 presents the median execution time of the three dense implementations. Error bars represent half of the measured interquartile range.

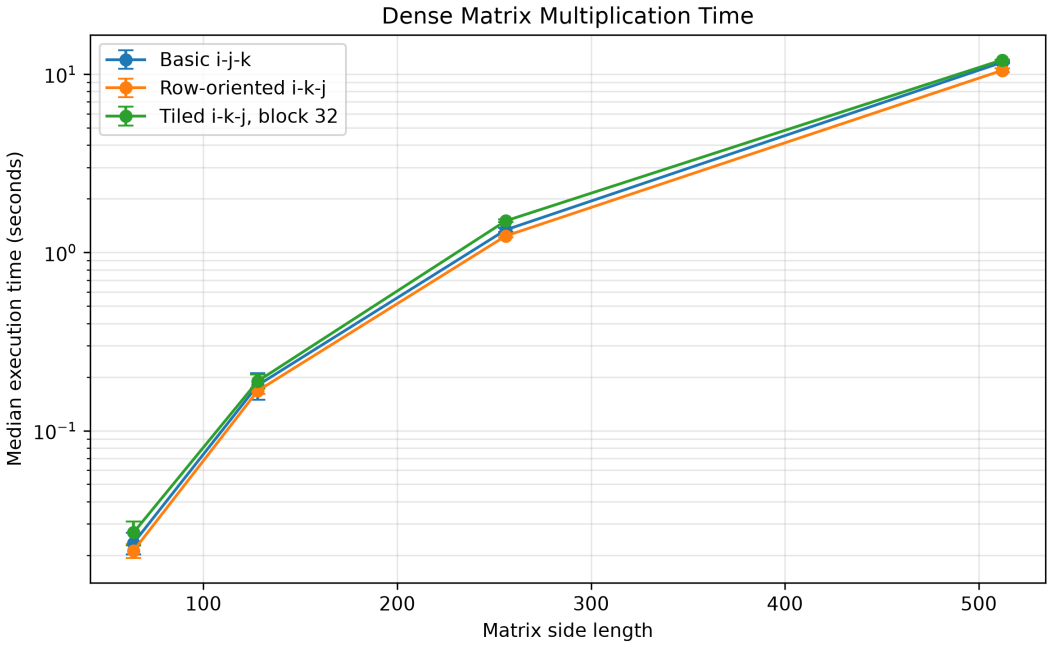


Figure 1: Median dense matrix multiplication time.

At 512×512, the basic implementation obtained a median execution time of 11.653921 seconds. The row-oriented implementation reduced this to 10.499886 seconds, corresponding to an approximate speedup of 1.11. The tiled implementation required 12.014305 seconds and was therefore approximately 3.1% slower than the baseline at the largest tested size.

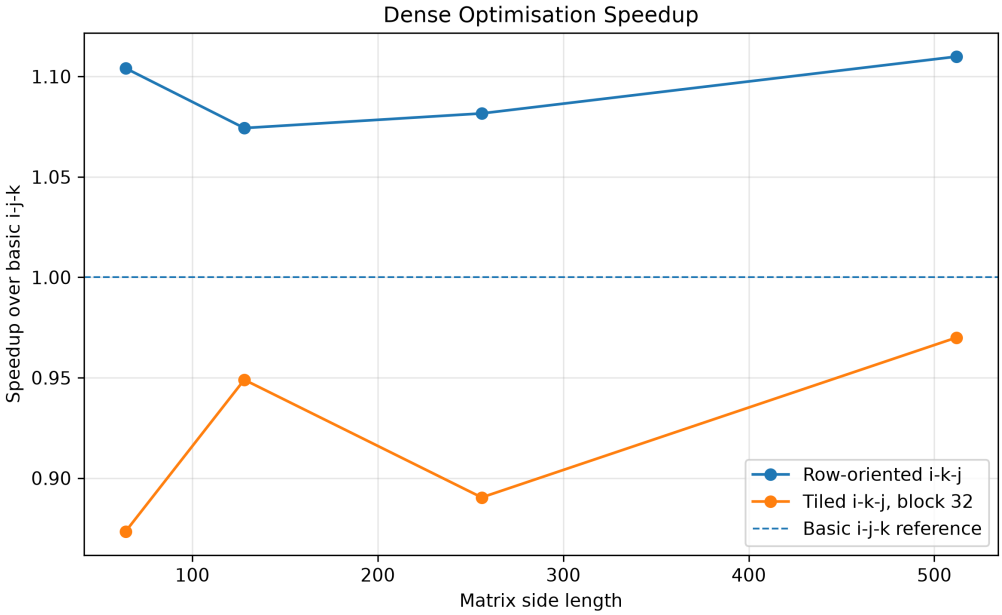


Figure 2: Speedup of dense optimisation techniques over the basic `i-j-k` implementation.

The row-oriented method became more beneficial as matrix size increased. In contrast, the tiled implementation did not outperform the baseline in the final configurations. Although blocking can improve locality in compiled implementations, the additional loop control performed by the Python interpreter introduced measurable overhead.

4.2 Memory Usage

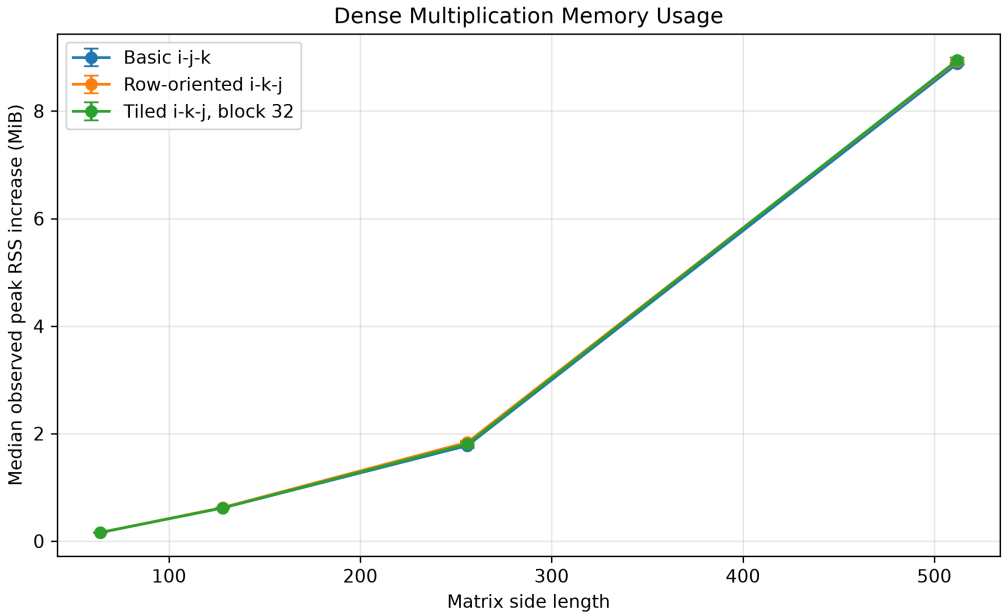


Figure 3: Observed peak RSS increase during dense multiplication.

At 512×512, the median observed peak RSS increases were 8.883 MiB for the basic implementation, 8.930 MiB for the row-oriented implementation and 8.945 MiB for the tiled implementation. These differences are small relative to the RSS measurement method. The three implementations retain the same two dense inputs and produce an output with the same dimensions, so no substantial memory advantage was observed.

5 Sparse Benchmark Results

5.1 Execution Time and Speedup

Figure 4 compares the matched dense method, custom CSR and SciPy CSR over the tested matrix sizes and sparsity levels.

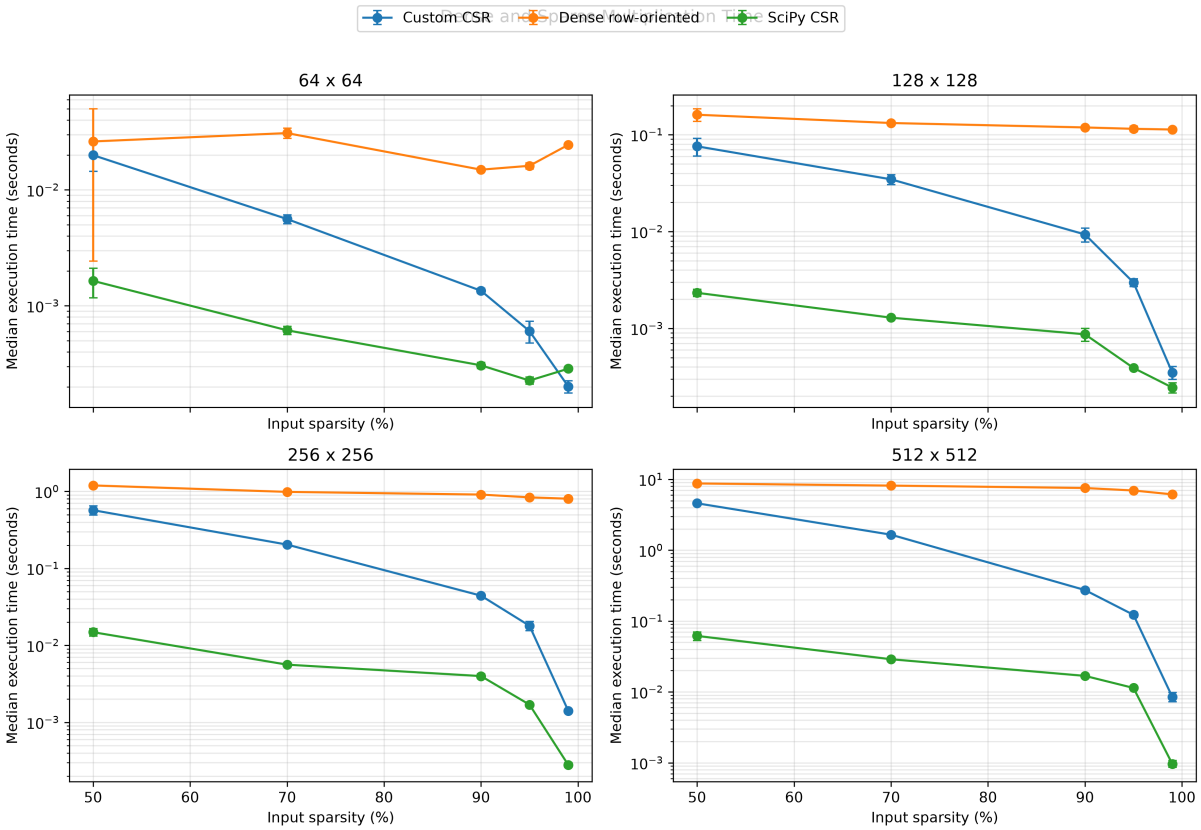


Figure 4: Dense and sparse multiplication time by input sparsity and matrix size.

Sparse multiplication became increasingly advantageous as the proportion of zero-valued inputs increased. The custom CSR implementation avoided multiplications associated with zero entries, while the dense method continued traversing all logical positions.

SciPy CSR produced the strongest performance in the larger controlled configurations. This result reflects the considerable difference between interpreted Python loops and optimised numerical-library operations. At very small execution times, isolated reversals between custom CSR and SciPy should not be interpreted as a general ranking because timer resolution, process activity and fixed library overhead represent a larger fraction of the measurement.

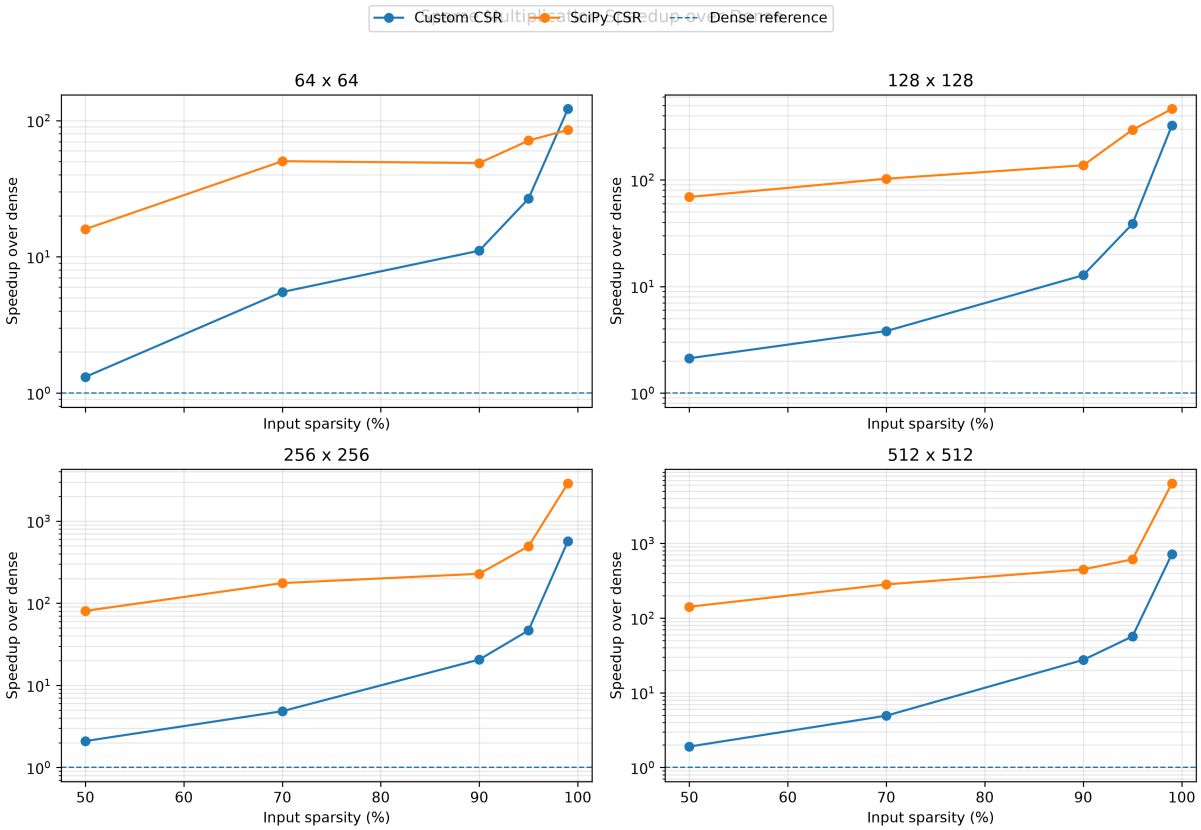


Figure 5: Speedup of the CSR implementations over the matched dense method.

5.2 Product Densification

Input sparsity alone does not determine the structure of the result. A multiplication combines paths through the shared inner dimension, so many sparse contributions may reach different output positions.

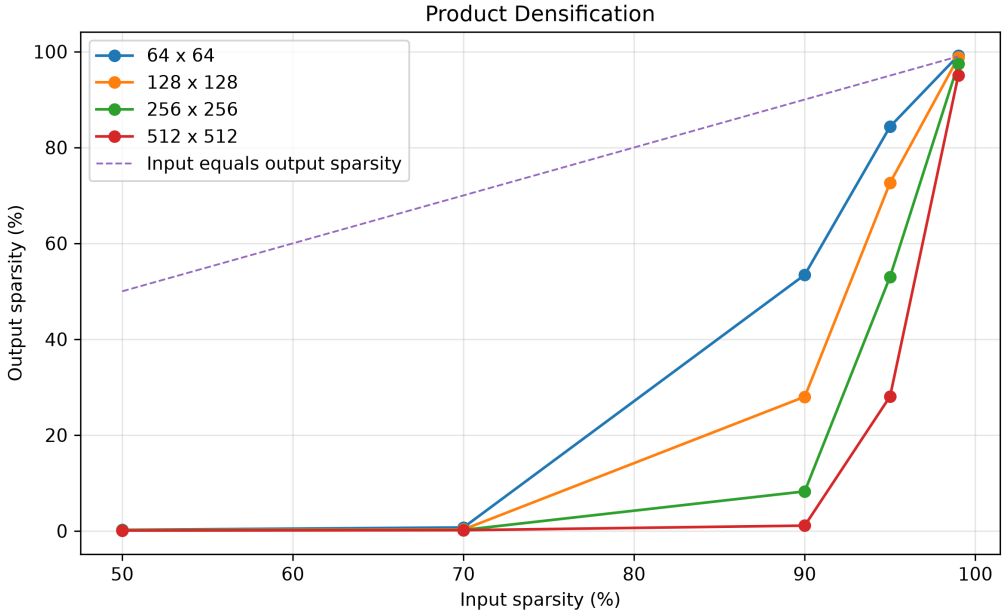


Figure 6: Relationship between input sparsity and output sparsity.

For example, at 256×256, input matrices with 50% sparsity produced an output with only approximately 0.1266% sparsity. Inputs with 70% sparsity produced approximately 0.2121% output sparsity. Even 90% sparse inputs produced an output with only approximately 8.2581% sparsity. At 99% input sparsity, the output remained highly sparse at approximately 97.4548%.

These results show that representation selection should consider expected output structure in addition to the sparsity of the operands.

5.3 Memory Usage

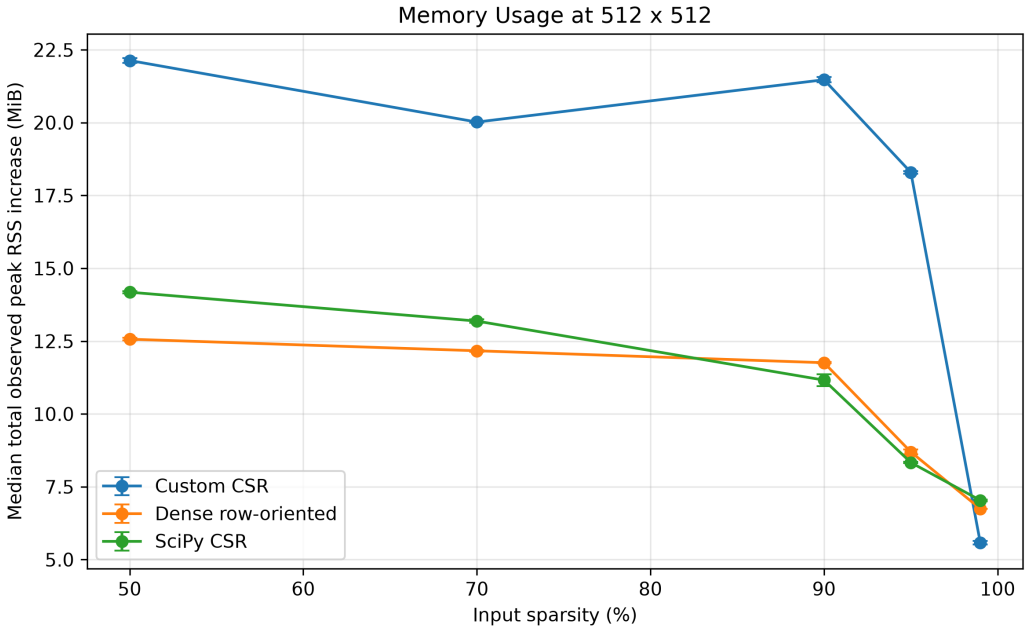


Figure 7: Observed total peak RSS increase for 512×512 configurations.

The sparse memory results include representation construction and multiplication within isolated processes. CSR storage becomes more favourable as the number of stored entries decreases. However, a dense or nearly dense product can reduce this benefit because the sparse result must also store an index for each output value.

6 Real-World Case Study: `mc2depi`

6.1 Dataset Characteristics

The `mc2depi` dataset is a general integer matrix distributed in Matrix Market coordinate format. It represents a two-dimensional Markov epidemic model and belongs to the Williams group of the SuiteSparse Matrix Collection.

The observed properties are shown in Table 1.

Table 1: Observed properties of the `mc2depi` matrix.

Property	Value
Rows	525,825
Columns	525,825
Stored nonzero entries	2,100,225
Logical matrix positions	276,491,930,625
Density	0.00075960%
Sparsity	99.99924040%
Minimum stored value	-3,559
Maximum stored value	2,025
Explicit stored zeros	0
CSR array storage	26.041 MiB
Theoretical dense `int64` storage	2,060.025 GiB

The dense storage estimate includes only one 64-bit integer per logical position. It excludes container and indexing overhead. A complete dense representation was never created.

6.2 Sparse Matrix-Vector Multiplication

The input vector contained 525,825 deterministic integer values. Five measured executions produced a median time of 0.0041597 seconds and an IQR of 0.0015522 seconds. All executions produced checksum 61,776 and 525,748 nonzero output values.

6.3 Sparse Matrix-Matrix Multiplication

The final real-world experiment computed:

$$C = A \times A,$$

where A is the complete `mc2depi` matrix.

Table 2: Final `mc2depi` sparse matrix-matrix results.

Metric	Value
Input nonzero entries	2,100,225
Output nonzero entries	5,245,952
Output sparsity	99.99810267%
Median multiplication time	0.0641652 s
Multiplication-time IQR	0.0045863 s
Input CSR storage	26.041 MiB
Output CSR storage	62.041 MiB
Output value sum	0
Output absolute-value sum	16,596,091,460,574

The output contained approximately 2.50 times as many stored entries as the input, but remained

extremely sparse. All five measured executions produced an identical SHA-256 digest over the complete canonical CSR arrays. This confirmed deterministic equivalence even though the algebraic sum of the output values was zero.

7 Discussion

7.1 Dense Optimisation Behaviour

The row-oriented loop order provided the strongest pure-Python dense result at the largest tested size. The improvement supports the expectation that sequential traversal of rows in the second operand is more suitable for Python list-of-lists storage than repeatedly accessing one logical column.

The tiled implementation did not outperform the baseline. Blocking introduced more loop boundaries, range calculations and control operations at Python level. The result demonstrates that an optimisation that benefits compiled numerical kernels does not automatically provide a benefit when its control structure is interpreted.

7.2 Sparse Performance and Crossover Behaviour

The sparse experiments support the hypothesis that CSR methods become more useful as sparsity increases. The dense method performs work for every logical combination, whereas CSR limits traversal to stored structural entries and their reachable contributions.

SciPy CSR achieved the best general sparse performance in the larger configurations. The custom implementation remained valuable as an explicit demonstration of CSR storage and sparse row accumulation, but its dictionary operations and Python loops prevented it from matching the optimised library.

7.3 Importance of Output Structure

One of the most important findings is that sparse operands may generate a dense or nearly dense product. The 50% and 70% synthetic configurations frequently produced almost fully populated outputs. A sparse representation can therefore lose part of its storage and computational advantage during multiplication.

The `mc2depi` product behaved differently. Despite increasing the number of stored entries from 2.10 million to 5.25 million, the output retained more than 99.998% sparsity. This structural behaviour allowed the full product to remain practical in CSR format.

7.4 Threats to Validity

The experiments were performed on one computer and one software environment. Results may differ with another processor, memory hierarchy, Python implementation or SciPy build.

The active Windows power plan was Balanced. Dynamic processor frequency, thermal conditions, background activity and file-system caching may have contributed to variation between runs.

RSS is a process-level metric rather than an exact measure of matrix storage. It includes interpreter and library memory, and periodic sampling may miss very short-lived allocations.

Only five measured executions were collected per configuration. Median and IQR reduce sensitivity to individual outliers, but a larger repetition count could provide stronger statistical characterisation.

The synthetic sparse generator controls the global number of nonzero entries but does not model every possible row distribution or structural pattern. Consequently, the observed crossover behaviour applies to the tested random structure and should not be generalised to all sparse matrices.

8 Conclusions

This study evaluated baseline and optimised dense multiplication, custom CSR multiplication, SciPy CSR multiplication and the real-world `mc2depi` dataset.

The row-oriented `i-k-j` implementation was the best pure-Python dense method at the largest tested size. It achieved an approximate 1.11 speedup over the basic implementation at 512×512. The tiled implementation did not improve performance because Python-level blocking overhead outweighed the observed locality benefit.

Sparse multiplication became increasingly advantageous as input sparsity increased. SciPy CSR provided the strongest general performance across the larger sparse configurations. However, input sparsity did not guarantee sparse output. Moderate input sparsity frequently produced dense or nearly dense products.

The `mc2depi` case study demonstrated the practical importance of sparse representation. Its CSR arrays required approximately 26.041 MiB, whereas a theoretical dense `int64` value array would require approximately 2,060.025 GiB. The complete sparse product $A \times A$ finished with a median time of 0.0641652 seconds and retained 99.99810267% sparsity.

The results show that representation and algorithm selection should depend on matrix dimensions, input sparsity, output structure, implementation environment and library overhead rather than on sparsity alone.

8.1 Future Work

Future work could investigate:

- alternative tile sizes and automatic block-size selection;
- NumPy dense multiplication as a separately classified library baseline;
- additional real-world matrices with different row distributions;
- multiplication between matrices with different sparsity structures;
- parallel and compiled custom CSR implementations;
- more extensive repetition counts and controlled power settings;
- hardware performance counters for cache misses and memory bandwidth;
- sparse output prediction before multiplication.

9 AI-Assistance Declaration

Generative AI assistance was used to support project planning, code review, test design, benchmark-protocol design, debugging and report structuring. All generated code and written content were reviewed, executed and validated by the author. All reported numerical results were obtained from the local experimental implementation and preserved raw measurements. No benchmark values were invented or copied from another student's report.

References

[1] T. A. Davis and Y. Hu, *The University of Florida Sparse Matrix Collection*, ACM Transactions on Mathematical Software, vol. 38, no. 1, 2011.

[2] SuiteSparse Matrix Collection, *Williams/mc2depi: Epidemiology, 2D Markov Model of Epidemic*, Matrix ID 2377.

[3] R. F. Boisvert, R. Pozo and K. A. Remington, *The Matrix Market Exchange Formats: Initial Design*, NISTIR 5935, National Institute of Standards and Technology, 1996.

[4] Python Software Foundation, *Python Standard Library: time — Time Access and Conversions*.

[5] The SciPy Community, *SciPy Sparse Matrix Documentation: Compressed Sparse Row Matrix*.

A Reproduction Commands

```
python -m pytest python/test/matrix -v

python -m python.src.matrix.benchmark.benchmark_dense \
  --profile full \
  --output results/raw/dense_time_full.csv

python -m python.src.matrix.benchmark.benchmark_dense_memory \
  --profile full \
  --output results/raw/dense_memory_full.csv

python -m python.src.matrix.benchmark.benchmark_sparse \
  --profile full \
  --output results/raw/sparse_time_full.csv

python -m python.src.matrix.benchmark.benchmark_sparse_memory \
  --profile full \
  --output results/raw/sparse_memory_full.csv

python -m python.src.matrix.benchmark.analyze_mc2depi

python -m python.src.matrix.benchmark.benchmark_mc2depi

python -m python.src.matrix.benchmark.benchmark_mc2depi_spgemm
```

```
python -m python.src.matrix.benchmark.process_results
```

```
python -m python.src.matrix.plots.plot_results
```

B Generated Figures

The following report figures are generated directly from the final processed benchmark results:

- `dense_execution_time.png`
- `dense_memory.png`
- `dense_speedup.png`
- `output_sparsity.png`
- `sparse_execution_time.png`
- `sparse_memory_512.png`
- `sparse_speedup.png`